ファイル一覧テーブル UI仕様書

1. 概要

1.1 コンポーネントの目的

ファイル一覧テーブルは、プロジェクト管理画面の「ファイル管理」タブ内に表示され、選択したプロジェクトのファイル・フォルダ一覧をテーブル形式で表示するコンポーネントです。ファイルのダウンロード、ビューア表示、フォルダナビゲーション機能を提供します。

1.2 使用場面

- プロジェクト管理画面 → 「ファイル管理」タブ選択時
- プロジェクト選択後に表示
- rootフォルダ または archiveフォルダの内容を表示

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証
- **組織分離**: マルチテナント（organizationId）

---

2. 画面構成

2.1 レイアウト構造

```
┌──────────────────────────────────────────────────────────┐
│ テーブルヘッダー                                          │
│ [種別] [ファイル名] [サイズ] [更新日時] [操作]            │
├──────────────────────────────────────────────────────────┤
│ [📁] archive (5)                                         │
├──────────────────────────────────────────────────────────┤
│ [📄] project-001.maiml  | 45.2 KB | 2時間前 | [DL] [表示] |
```

```
├──────────────────────────────────────────────────────────┤
│ [ 📄 ] result.csv          │ 12.8 KB │ 1日前    │ [DL] [表示] │
├──────────────────────────────────────────────────────────┤
│ [ 📊 ] data.xlsx           │ 128 KB  │ 3日前    │ [DL] [表示] │
└──────────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 テーブルカラム

1. 種別（type）

- 幅: 50px
- 表示: ファイル/フォルダタイプに応じたアイコン
- 中央揃え

2. ファイル名（name）

- ソート可能（日本語対応）
- フォルダの場合: 太字 + オレンジ色 + ファイル数表示
- ツールチップ: ファイルパス表示

3. サイズ（size）

- 幅: 120px
- ソート可能
- フォルダの場合: 非表示
- フォーマット: `formatFileSize()`（KB, MB単位）

4. 更新日時（lastModified）

- 幅: 150px
- ソート可能
- フォルダの場合: 非表示
- フォーマット: `formatRelativeTime()`（相対時刻表示）

5. 操作（actions）

- 幅: 220px

- 中央揃え
- フォルダの場合: 非表示
- ファイルの場合: ダウンロードボタン + ビューアボタン

2.2.2 ファイルタイプアイコン

ファイルタイプ	アイコン	色
フォルダ	FolderOpenOutlined	オレンジ（`token.colorWarning`）
MAIML（.maiml）	FileTextOutlined	青（`token.colorInfo`）
PDF（.pdf）	FileTextOutlined	赤（`token.colorError`）
Excel（.xlsx）	TableOutlined	緑（`token.colorSuccess`）
CSV（.csv）	TableOutlined	グレー（`token.colorTextSecondary`）
アーカイブ	FolderOpenOutlined	オレンジ（`token.colorWarning`）
その他	FileOutlined	グレー（`token.colorTextSecondary`）

2.2.3 操作ボタン

ダウンロードボタン:

- アイコン: DownloadOutlined
- サイズ: small
- タイプ: default
- ローディング状態: ダウンロード中は spinner表示
- 無効化: 他のファイルダウンロード中は全ボタン無効

ビューアボタン:

- アイコン: EyeOutlined
- サイズ: small
- タイプ: primary
- 背景色: 緑（`token.colorSuccess`）
- クリック: ViewRendererDialogを開く

2.3 Empty状態

プロジェクトにファイルが存在しない場合:

```
<Empty
  description={t('projectManagement.fileManagement.emptyState')}
  image={Empty.PRESENTED_IMAGE_SIMPLE}
/>
```

---

3. 機能仕様

3.1 ファイル一覧表示

3.1.1 表示内容

```
<Table
  dataSource={files}
  columns={columns}
  rowKey="id"
  size="small"
  pagination={false}
/>
```

3.1.2 データソース

- `files`: 親コンポーネント（ProjectManagement）から渡されるファイル一覧
- rootフォルダ: プロジェクト直下のファイル + archiveフォルダ（存在する場合）
- archiveフォルダ: アーカイブされた過去バージョンのファイル + 戻るボタン

3.2 ファイルダウンロード

3.2.1 処理フロー

```
const handleDownloadFile = async (file: ProjectFile) => {
  try {
    setDownloadingFileId(file.id);

    // ファイルパスが存在するか確認
    if (!file.path || file.path.trim() === '') {
      throw new Error(t('projectManagement.fileManagement.messages.filePathMissing'));
    }

    // ファイルキーベースで直接ダウンロード（プロキシ方式）
    await getFileDownloadUrlByKey(file.path);

    // 成功メッセージ表示
    message.success(
      t('projectManagement.fileManagement.messages.downloadStarted').replace('{{fileName}}',
file.name)
    );
  } catch (error) {
    console.error('ファイルダウンロードエラー:', error);
    const errorMsg = error instanceof Error ? error.message : '不明なエラー';
    message.error(
      t('projectManagement.fileManagement.messages.downloadError').replace('{{error}}',
errorMsg)
    );
  } finally {
    setDownloadingFileId(null);
  }
};
```

3.2.2 UI動作

1. ダウンロードボタンをクリック
2. ボタンがローディング状態になる（spinner表示）
3. 他のダウンロードボタンが無効化される
4. バックエンドAPIを呼び出し（プロキシ方式）
5. 成功: ファイルダウンロード開始 + 成功メッセージ
6. 失敗: エラーメッセージ表示
7. ローディング状態解除

3.2.3 プロキシ方式ダウンロード

```
await getFileDownloadUrlByKey(file.path);
```

- バックエンド経由で直接ダウンロード
- S3署名付きURLを内部で生成・リダイレクト
- ブラウザのダウンロード機能でファイル保存

3.3 ビューア表示

3.3.1 処理フロー

```
<Button
  type="primary"
  size="small"
  icon={<EyeOutlined />}
  onClick={(e) => {
    e.stopPropagation();
    onViewFile?.(record);
  }}
>
  ビューア
</Button>
```

3.3.2 UI動作

1. ビューアボタンをクリック
2. 親コンポーネント（ProjectManagement）の `handleViewFile` を呼び出し
3. ViewRendererDialogが開く
4. ファイル内容を表示

3.4 フォルダクリック

3.4.1 処理フロー

```
onRow={(record) => ({
  onClick: () => {
    if (record.isFolder) {
      onFolderClick?.(record.id);
    } else {
      onFileClick?.(record);
    }
  },
  style: {
    cursor: record.isFolder || onFileClick ? 'pointer' : 'default',
  },
})}
```

3.4.2 UI動作

- フォルダ行をクリック: `onFolderClick` コールバック呼び出し
  - archiveフォルダ → archiveフォルダ内のファイル一覧を表示
  - backフォルダ（"↑ 戻る"） → rootフォルダに戻る
- ファイル行をクリック: `onFileClick` コールバック呼び出し（現在未使用）

3.5 ソート機能

3.5.1 ソート可能カラム

ファイル名（name）:

```
sorter: (a, b) => a.name.localeCompare(b.name, 'ja')
```

- 日本語対応のアルファベット順ソート

サイズ（size）:

```
sorter: (a, b) => a.size - b.size
```

- バイト数による数値ソート

更新日時（lastModified）:

```
sorter: (a, b) => new Date(a.lastModified).getTime() - new Date(b.lastModified).getTime()
```

- タイムスタンプによる日時ソート

3.6 多言語対応

3.6.1 翻訳キー

```
const { t } = useLanguage();

// テーブルヘッダー
t('projectManagement.fileManagement.table.type')         // "種別"
t('projectManagement.fileManagement.table.fileName')     // "ファイル名"
t('projectManagement.fileManagement.table.size')         // "サイズ"
t('projectManagement.fileManagement.table.lastModified') // "更新日時"
t('projectManagement.fileManagement.table.actions')      // "操作"

// ボタン
t('projectManagement.fileManagement.buttons.download')   // "ダウンロード"
t('projectManagement.fileManagement.buttons.viewer')     // "ビューア"

// メッセージ
t('projectManagement.fileManagement.messages.downloadStarted')  // "{{fileName}}のダウンロードを開始しました"
t('projectManagement.fileManagement.messages.downloadError')    // "ダウンロードエラー: {{error}}"
t('projectManagement.fileManagement.messages.filePathMissing')  // "ファイルパスが見つかりません"

// Empty状態
t('projectManagement.fileManagement.emptyState')         // "ファイルがありません"
```

---

4. データ仕様

4.1 Props定義

```typescript
interface FileListTableProps {
  /** 表示するファイル・フォルダー一覧 */
  files: ProjectFile[];
  /** プロジェクトID */
  projectId: string;
  /** ファイルクリック時のコールバック（現在未使用） */
  onFileClick?: (file: ProjectFile) => void;
  /** フォルダクリック時のコールバック */
  onFolderClick?: (folderId: string) => void;
  /** ビューアボタンクリック時のコールバック */
  onViewFile?: (file: ProjectFile) => void;
}
```

Props説明:

Prop名	型	必須	説明
files	`ProjectFile[]`	✅	表示するファイル・フォルダー一覧
projectId	`string`	✅	プロジェクトID（現在未使用）
onFileClick	`(file: ProjectFile) => void`	-	ファイル行クリック時のコールバック（現在未使用）
onFolderClick	`(folderId: string) => void`	-	フォルダクリック時のコールバック
onViewFile	`(file: ProjectFile) => void`	-	ビューアボタンクリック時のコールバック

4.2 State定義

```typescript
// ダウンロード中のファイルID
const [downloadingFileId, setDownloadingFileId] = useState<string | null>(null);
```

State説明:

State名	型	初期値	説明

downloadingFileId	`string | null`	`null`	ダウンロード中のファイルID。nullの場合はダウンロード中のファイルなし

4.3 ProjectFile型定義

```
interface ProjectFile {
  id: string;              // ファイルID
  name: string;            // ファイル名
  path: string;            // ファイルパス（S3キー）
  size: number;            // ファイルサイズ（バイト）
  lastModified: string;    // 更新日時（ISO8601形式）
  type: string;            // ファイルタイプ（maiml, pdf, excel, csv, folder等）
  isFolder: boolean;       // フォルダかどうか
  fileCount?: number;      // フォルダの場合のファイル数
}
```

4.4 Context連携

```
// LanguageContext
const { t } = useLanguage();

// Ant Design Theme
const { token } = theme.useToken();
```

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
1	getFileDownloadUrlByKey	GET	`/api/v1/files/download-proxy`	✅ 実装済み	ファイルをプロキシ経由でダウンロード

5.2 API呼び出し例

5.2.1 ファイルダウンロード（プロキシ方式）

```
await getFileDownloadUrlByKey(file.path);
```

Request:

```
GET /api/v1/files/download-proxy?fileKey={file.path}
```

Response:

- 成功: ファイルストリームをブラウザにリダイレクト
- 失敗: エラーレスポンス

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
ファイルパスが空	message.error('ファイルパスが見つかりません')
ダウンロード失敗	message.error('ダウンロードエラー: {エラーメッセージ}')
API呼び出し失敗	エラーメッセージ表示 + コンソールログ出力

6.2 エラー処理コード

```
try {
  setDownloadingFileId(file.id);

  if (!file.path || file.path.trim() === '') {
    throw new Error(t('projectManagement.fileManagement.messages.filePathMissing'));
  }
```

```
  await getFileDownloadUrlByKey(file.path);

message.success(t('projectManagement.fileManagement.messages.downloadStarted').replace('{{fileNa
file.name));
} catch (error) {
  console.error('ファイルダウンロードエラー:', error);
  const errorMsg = error instanceof Error ? error.message : '不明なエラー';

message.error(t('projectManagement.fileManagement.messages.downloadError').replace('{{error}}',
errorMsg));
} finally {
  setDownloadingFileId(null);
}
```

---

7. 実装ファイル

7.1 メインファイル

- **パス**: `frontend/src/components/project/FileListTable.tsx`
- **行数**: 約200行
- **エクスポート**: `FileListTable`（default export）

7.2 関連ファイル

7.2.1 型定義

- `frontend/src/types/file.ts` - ProjectFile型

7.2.2 APIサービス

- `frontend/src/services/fileAccessService.ts` - getFileDownloadUrlByKey()

7.2.3 ユーティリティ

- `frontend/src/utils/formatters.ts` - formatFileSize(), formatRelativeTime()

7.2.4 Context

- `frontend/src/contexts/LanguageContext.tsx` - 多言語対応

7.3 依存ライブラリ

- React 18.2.0
- Ant Design v6 (Table, Button, Tooltip, Space, Empty, message, theme)
- @ant-design/icons (FileTextOutlined, FileOutlined, TableOutlined, FolderOpenOutlined, DownloadOutlined, EyeOutlined)

---

8. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を再作成	Cline

---

9. 補足事項

9.1 フォーマット関数

9.1.1 formatFileSize

```
formatFileSize(bytes: number): string
```

- 入力: バイト数
- 出力: KB, MB, GB単位の文字列
- 例: `45200` → `"45.2 KB"`, `1048576` → `"1 MB"`

9.1.2 formatRelativeTime

```
formatRelativeTime(isoDate: string): string
```

- 入力: ISO8601形式の日時文字列
- 出力: 相対時刻の文字列
- 例: `"2026-03-11T07:00:00Z"` → `"2時間前"`, `"2026-03-10T09:00:00Z"` → `"1日前"`

9.2 テーブルスタイリング

```
<Table
  dataSource={files}
  columns={columns}
  rowKey="id"
  size="small"           // 小さめのサイズ
  pagination={false}     // ページネーション無効
  onRow={(record) => ({
    onClick: () => { /* ... */ },
    style: {
      cursor: record.isFolder || onFileClick ? 'pointer' : 'default',
    },
  })}
/>
```

9.3 イベントバブリング制御

```
onClick={(e) => {
  e.stopPropagation();  // 行クリックイベントの伝播を停止
  handleDownloadFile(record);
}}
```

- ボタンクリック時に行クリックイベントが発火しないようにする

9.4 ファイルタイプ判定

```
const getFileIcon = (file: ProjectFile) => {
  if (file.isFolder) {
    return <FolderOpenOutlined />;
  }

  switch (file.type) {
    case 'maiml': return <FileTextOutlined />;
    case 'pdf': return <FileTextOutlined />;
    case 'excel': return <TableOutlined />;
    case 'csv': return <TableOutlined />;
    case 'archive': return <FolderOpenOutlined />;
    default: return <FileOutlined />;
  }
};
```

9.5 フォルダ表示の特徴

- **太字表示**: `fontWeight: token.fontWeightStrong`
- **オレンジ色**: `color: token.colorWarning`
- **ファイル数表示**: `archive (5)` のように括弧内に件数表示
- **サイズ・更新日時非表示**: フォルダの場合は空欄
- **操作ボタン非表示**: フォルダの場合はボタンなし

9.6 ダウンロード状態管理

- **単一ファイルダウンロード**: 一度に1ファイルのみダウンロード可能
- **ローディング表示**: ダウンロード中のボタンにspinner表示
- **他ボタン無効化**: `disabled={downloadingFileId !== null}` で他のダウンロードボタンを無効化

9.7 プロキシ方式ダウンロードの利点

- クライアント側で署名付きURL生成不要
- セキュリティ向上（S3への直接アクセス不要）

- ダウンロード進捗の追跡が可能
- エラーハンドリングの一元化

9.8 アクセシビリティ

- ツールチップでファイルパス表示
- クリック可能要素にカーソル変更
- ソート機能でデータ整理
- Empty状態の明確な表示